

UP LT GRADE • COMPUTER SCIENCE MAINS

Software Engineering Complete Descriptive-Exam Notes

Simple, Clear, Exam-Ready — With Diagrams

Foundations: Definition & SDLC Models • CMM • Software Quality • Metrics & Measurement

Planning & Design: Requirements Analysis & Specification • Project Planning • Software Architecture • Design & Implementation

Verification: Software Testing • Software Reliability

Prepared for descriptive-exam revision • 45-day study plan

Table of Contents

1. Introduction to Software Engineering	Page 3
2. Software Development Life Cycle (SDLC) Models	Page 4
3. Capability Maturity Model (CMM)	Page 7
4. Software Quality	Page 8
5. Role of Metrics and Measurement	Page 10
6. Requirements Analysis and Specification	Page 11
7. Software Project Planning	Page 13
8. Software Architecture	Page 15
9. Software Design and Implementation	Page 17
10. Software Testing	Page 19
11. Software Reliability	Page 21

1. Introduction to Software Engineering

In Simple Words

Software Engineering is the disciplined, systematic approach to designing, developing, testing, and maintaining software — applying engineering principles (planning, process, quality control) instead of just writing code ad-hoc.

Definition

IEEE defines Software Engineering as: "The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software."

Characteristics of Software (vs Hardware)

- **Software is developed/engineered, not manufactured:** Unlike hardware, there's no assembly-line production cost — the cost is in DESIGN, not replication.
- **Software doesn't "wear out":** It doesn't degrade physically, but it does suffer from repeated CHANGE (maintenance), which introduces defects over time — this is sometimes visualized as a rising "defect rate" curve instead of hardware's classic bathtub curve.
- **Most software is custom-built:** Though reusable components/libraries are increasingly used, software is still largely tailored to specific requirements.

Goals of Software Engineering

- Deliver software that meets user requirements (Correctness)
- Deliver it on time and within budget (Efficiency of process)
- Ensure it is maintainable, reliable, and reusable (Quality attributes)
- Manage complexity through systematic processes and tools

Exam Tip

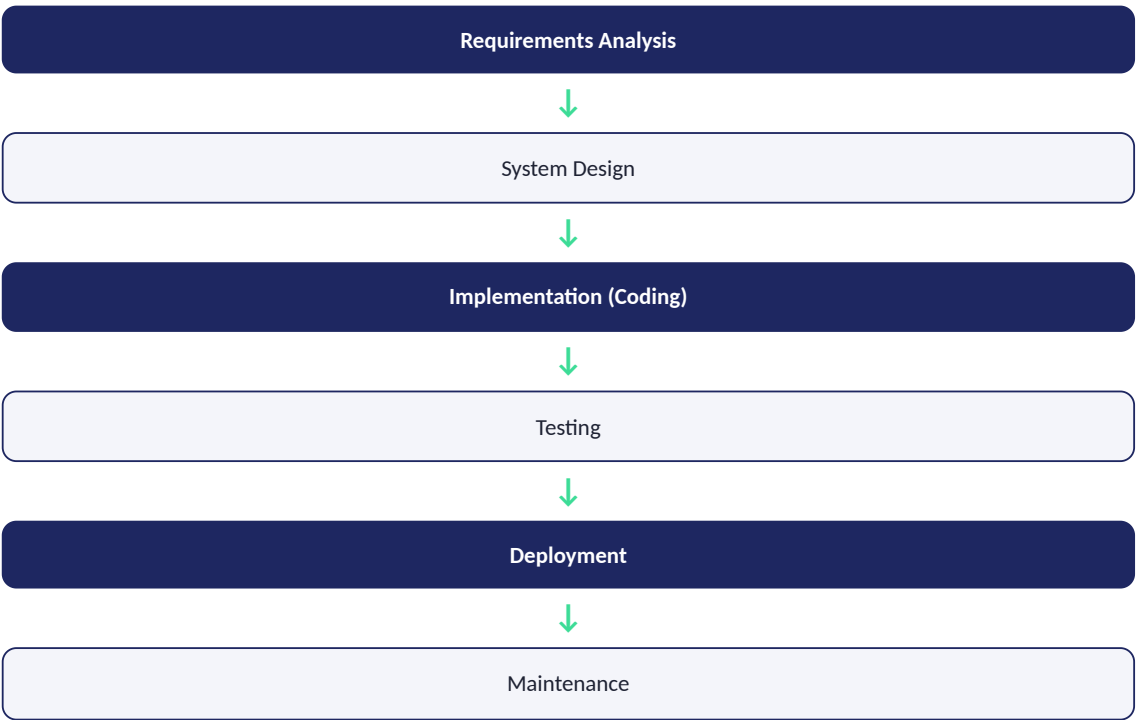
Always explicitly contrast software with hardware characteristics ("doesn't wear out but does deteriorate through change") when defining software — this comparison is what shows genuine conceptual understanding, not just a memorized definition.

2. Software Development Life Cycle (SDLC) Models

In Simple Words

An SDLC Model is a structured sequence of phases (or a strategy for repeating them) that a software project follows from initial idea to final deployment and maintenance. Different models suit different types of projects.

Diagram: Waterfall Model (Linear, Sequential)



Each phase must FULLY complete before the next begins — no going back without significant cost.

Waterfall Model — Pros and Cons

Advantages	Disadvantages
Simple, easy to understand and manage	Very inflexible — hard to accommodate changing requirements
Clear milestones and documentation at each phase	Working software only available at the very end (high risk)
Good for well-understood, stable requirements	Errors found late are very expensive to fix

Other Major SDLC Models

- **Prototype Model:** A working mock-up (prototype) is built quickly based on initial requirements, shown to the user for feedback, and refined iteratively before full development — reduces the risk of misunderstanding requirements.
- **Spiral Model:** Combines iterative development with the systematic aspects of the Waterfall model, adding an explicit RISK ANALYSIS phase in every loop ("spiral") — each loop produces a more refined version of the software. Best for large, high-risk projects.
- **Iterative/Incremental Model:** The system is built and delivered in small INCREMENTS, each adding functionality, rather than all at once — allows earlier partial delivery and feedback.
- **Agile Model:** Emphasizes short iterations ("sprints"), continuous customer collaboration, and adapting to changing requirements over rigid planning — e.g., Scrum, XP (Extreme Programming).

Spiral Model — Four Quadrant Activities (per loop)

Quadrant	Activity
1	Determine objectives, alternatives, and constraints for this iteration
2	Evaluate alternatives; identify and resolve RISKS (the defining feature of this model)
3	Develop and verify the next-level product (design, code, test)
4	Plan the next iteration/phase

Exam Tip

Always draw the Waterfall diagram (it's the most-requested SDLC diagram) AND explicitly name Spiral's unique feature — Risk Analysis — since that single word is what distinguishes it from every other iterative model in comparison questions.

3. Capability Maturity Model (CMM)

In Simple Words

CMM is a framework that measures how MATURE (organized, disciplined, and consistent) a software organization's development PROCESS is — not the software itself, but the process used to build it. It has 5 levels, each building on the previous.

Diagram: The Five CMM Levels



Key Point for Each Level

Level	Key Characteristic
1 — Initial	No defined process; outcomes are unpredictable and depend on individual heroics.
2 — Repeatable	Basic project tracking (cost, schedule) exists; past successes CAN be repeated for similar projects.
3 — Defined	A standard, documented process exists across the WHOLE organization, not just per-project.
4 — Managed	Process and product quality are measured quantitatively (metrics-driven).
5 — Optimizing	Continuous improvement is built in, using quantitative feedback from the process itself.

Exam Tip

Always draw the 5-level pyramid (as above) and give the ONE-WORD defining characteristic per level (Initial=chaotic, Repeatable=trackable, Defined=standardized, Managed=measured, Optimizing=improving) — these five words alone form a complete, scoring answer skeleton.

4. Software Quality

In Simple Words

Software Quality means the software meets specified requirements AND satisfies implicit expectations of good practice — not just "does it work," but "does it work well, reliably, and maintainably."

McCall's Quality Factors (Triangle of Quality)

Category	Factors	Focus
Product Operation	Correctness, Reliability, Efficiency, Integrity, Usability	How well it runs NOW
Product Revision	Maintainability, Flexibility, Testability	How easily it can be CHANGED
Product Transition	Portability, Reusability, Interoperability	How well it adapts to NEW environments

Key Quality Attributes Explained

- **Correctness:** Extent to which software meets its specified requirements.
- **Reliability:** Extent to which software performs its intended function without failure, under stated conditions.
- **Efficiency:** Amount of computing resources (time, memory) required for a given function.
- **Maintainability:** Effort required to locate and fix a defect or make a modification.
- **Portability:** Effort required to transfer software from one environment/platform to another.

ISO 9126 Quality Model (Alternative Standard)

Defines 6 quality characteristics: Functionality, Reliability, Usability, Efficiency, Maintainability, Portability — largely overlapping with McCall's model but standardized internationally.

Exam Tip

When listing quality factors, group them using McCall's THREE categories (Operation/Revision/Transition) rather than a flat list of 11 terms — the grouping itself demonstrates structured understanding and is easier to write correctly within the 125/200-word limit.

5. Role of Metrics and Measurement

In Simple Words

You cannot manage or improve what you don't measure. Software Metrics are quantitative measures used to track project progress, code quality, and process effectiveness objectively, instead of relying on guesswork.

Types of Metrics

- **Product Metrics:** Measure characteristics of the software product itself — e.g., Lines of Code (LOC), Cyclomatic Complexity, Function Points.
- **Process Metrics:** Measure characteristics of the DEVELOPMENT PROCESS — e.g., defect removal efficiency, average time to fix a bug.
- **Project Metrics:** Measure project-specific factors — e.g., number of developers, cost, schedule variance.

Common Size/Complexity Metrics

Metric	What It Measures
LOC (Lines of Code)	Simple size measure — total lines written; easy but inconsistent across languages/styles.
Function Points (FP)	Measures functionality delivered to the user, independent of the programming language — based on inputs, outputs, inquiries, files, interfaces.
Cyclomatic Complexity	Measures the number of independent paths through a program's control flow — higher value means more complex, harder-to-test code.

Cyclomatic Complexity Formula

$$V(G) = E - N + 2P$$

where E = number of edges, N = number of nodes, P = number of connected components (usually 1 for a single program) in the program's control flow graph.

Exam Tip

If asked to calculate Cyclomatic Complexity, always draw the control flow graph FIRST, then apply the formula — examiners specifically check whether you can construct the graph, not just recall/apply the formula blindly.

6. Requirements Analysis and Specification

In Simple Words

Before building any software, you must clearly understand and document WHAT the software must do. Requirements Analysis is the process of gathering, analyzing, and documenting these needs, resulting in a formal document called the SRS (Software Requirements Specification).

Requirements Engineering Process



Types of Requirements

- **Functional Requirements:** WHAT the system should DO — specific features, functions, behaviors (e.g., "system shall allow user login").
- **Non-Functional Requirements:** HOW WELL the system performs — quality attributes like performance, security, usability, scalability (e.g., "system shall respond within 2 seconds").

Characteristics of a Good SRS

- **Correct:** Accurately reflects actual user needs.
- **Unambiguous:** Only one possible interpretation for each requirement.
- **Complete:** All requirements are included, nothing missing.
- **Consistent:** No requirement contradicts another.
- **Verifiable:** Possible to check/test whether the requirement was met.
- **Traceable:** Each requirement can be traced to its origin and to the design/code that implements it.

Exam Tip

Always distinguish Functional vs Non-Functional requirements with a concrete example of EACH — this is one of the most frequently repeated "differentiate" questions in Software Engineering papers.

7. Software Project Planning

In Simple Words

Before coding begins, a project must be **PLANNED** — estimating how much it will cost, how long it will take, and what risks could derail it. Good planning prevents most large-scale project failures.

Software Cost/Effort Estimation — COCOMO Model

Basic COCOMO Formula

Effort = $a \times (\text{KLOC})^b$ person-months

where KLOC = thousands of lines of code (estimated size), and a, b are constants depending on project type:

Project Type	a	b
Organic (small, simple)	2.4	1.05
Semi-Detached (medium)	3.0	1.12
Embedded (complex, tight constraints)	3.6	1.20

Project Scheduling Tools

- **Gantt Chart:** A bar chart showing each task's start/end date and duration on a timeline — good for visualizing overlap and progress, but doesn't clearly show task **DEPENDENCIES**.
- **PERT Chart (Program Evaluation Review Technique):** A network diagram showing task dependencies and estimated durations, using optimistic/pessimistic/most-likely time estimates — good for identifying the Critical Path.
- **CPM (Critical Path Method):** Identifies the **CRITICAL PATH** — the longest sequence of dependent tasks that determines the **MINIMUM** possible project duration; delays on this path directly delay the whole project.

Risk Management

- **Risk Identification:** Listing potential risks (technical, schedule, budget, personnel).
- **Risk Analysis:** Assessing probability and impact of each identified risk.
- **Risk Planning:** Developing strategies to avoid, mitigate, or prepare contingencies for each significant risk.
- **Risk Monitoring:** Continuously tracking risks throughout the project lifecycle.

Exam Tip

Always state the COCOMO formula with clearly labeled a, b constants **AND** explain what KLOC means — numerical COCOMO questions expect you to plug in given values into this exact formula, so memorize it precisely.

8. Software Architecture

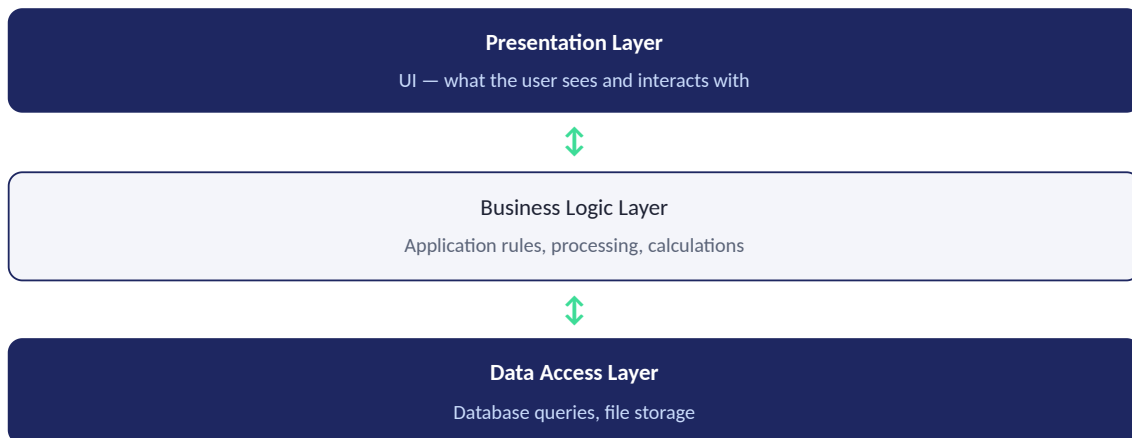
In Simple Words

Software Architecture is the high-level structure of a system — how major components are organized and how they interact — decided BEFORE detailed design begins. It's the "blueprint" that guides all further design decisions.

Common Architectural Styles

- **Layered (N-Tier) Architecture:** System organized into horizontal layers (e.g., Presentation, Business Logic, Data Access), where each layer only communicates with the layer directly below/above it — promotes separation of concerns.
- **Client-Server Architecture:** System split into a Client (requests services) and Server (provides services), communicating over a network — the dominant style for web applications.
- **MVC (Model-View-Controller):** Separates an application into Model (data/logic), View (UI/presentation), and Controller (handles user input, coordinates Model and View) — widely used in web frameworks.
- **Pipe-and-Filter Architecture:** Data flows through a series of processing components ("filters") connected by "pipes," each filter transforming the data before passing it to the next — e.g., Unix command-line pipelines.
- **Microservices Architecture:** System built as a collection of small, independently deployable services, each responsible for one specific business capability, communicating via APIs.

Diagram: Layered Architecture



Exam Tip

Always name the SPECIFIC architectural style being used in any given scenario question (don't just say "good architecture") and briefly justify WHY it fits (e.g., "Layered suits this because concerns are clearly separable") — vague architectural discussion loses marks to specific, justified choices.

9. Software Design and Implementation

In Simple Words

Design translates WHAT the system must do (requirements) into HOW it will be built internally (modules, data structures, algorithms). Implementation is then actually WRITING the code based on that design.

Key Design Concepts — Coupling and Cohesion

Concept	Definition	Goal
Coupling	Degree of interdependence BETWEEN different modules	Want LOW coupling (modules should be as independent as possible)
Cohesion	Degree to which elements WITHIN a single module belong together / work toward one purpose	Want HIGH cohesion (each module should do one well-defined job)

Golden Rule of Good Design

Aim for **Low Coupling, High Cohesion** — modules that are internally focused (high cohesion) and loosely connected to each other (low coupling) are easier to understand, test, reuse, and maintain independently.

Design Approaches

- **Structured Design:** Top-down, function-oriented decomposition of the system into modules — uses tools like Data Flow Diagrams (DFDs) and Structure Charts.
- **Object-Oriented Design (OOD):** Organizes the system around OBJECTS (combining data and behavior) — uses concepts like Encapsulation, Inheritance, Polymorphism, and is documented using UML diagrams (Class, Sequence, Use Case diagrams).

Implementation Best Practices

- **Coding Standards:** Consistent naming conventions, indentation, and commenting for readability and team consistency.
- **Code Reviews:** Peer examination of code before integration, to catch defects early and share knowledge.
- **Version Control:** Systematically tracking code changes (e.g., Git), enabling collaboration and rollback if needed.

Exam Tip

"Low Coupling, High Cohesion" is THE single most quoted phrase in Software Design questions — always use it explicitly (not a paraphrase) since examiners specifically look for this exact terminology.

10. Software Testing

In Simple Words

Testing is the process of executing software to find defects BEFORE the end user does. It verifies that the software behaves as expected across many levels, from individual functions up to the complete integrated system.

Diagram: V-Model (Testing Mapped to Development Phases)



The V-Model pairs EACH development phase with a corresponding testing level, planned from the very start (not as an afterthought).

Levels of Testing

Level	Focus
Unit Testing	Tests the SMALLEST piece — a single function/method/class in isolation.
Integration Testing	Tests how MULTIPLE units work together when combined.
System Testing	Tests the COMPLETE, integrated system against overall requirements.
Acceptance Testing	Validates the system against USER needs — often performed BY the customer before final sign-off.

Black-Box vs White-Box Testing

Aspect	Black-Box Testing	White-Box Testing
Knowledge of internal code	NOT required — tests based only on inputs/outputs	REQUIRED — tests based on internal logic/structure
Also called	Functional / Behavioral testing	Structural / Glass-box testing
Example Technique	Equivalence Partitioning, Boundary Value Analysis	Statement Coverage, Branch Coverage, Path Coverage

Exam Tip

Always draw the V-Model diagram when asked about testing levels — it visually PROVES the pairing between development and testing phases, which is the exact insight examiners want beyond just listing testing types.

11. Software Reliability

In Simple Words

Software Reliability is the probability that software will operate WITHOUT FAILURE for a specified period, under specified conditions — essentially, how much you can TRUST the software to keep working correctly.

Key Reliability Concepts

- **Failure:** The software's observable, external behavior deviates from what's expected/specified.
- **Fault (Defect/Bug):** The internal, root-cause flaw in the code that CAUSES a failure when triggered.
- **Error:** A human MISTAKE made during development that introduces a fault.
- **MTTF (Mean Time To Failure):** Average time the software runs before a failure occurs.
- **MTTR (Mean Time To Repair):** Average time taken to fix a failure once it occurs.
- **MTBF (Mean Time Between Failures):** $MTBF = MTTF + MTTR$ — average time between consecutive failures.

Key Relationship

$$\text{Availability} = \text{MTTF} / (\text{MTTF} + \text{MTTR}) \times 100\%$$

This shows that reliability isn't just about avoiding failure (MTTF) — how QUICKLY you recover (MTTR) is equally important for overall system availability.

Improving Reliability

- **Fault Avoidance:** Preventing faults from being introduced in the first place (good design, coding standards, reviews).
- **Fault Tolerance:** Designing the system to continue operating correctly even when a fault DOES occur (redundancy, error handling).
- **Fault Detection & Removal:** Thorough testing to find and fix faults before release.

Exam Tip

Always clearly distinguish Error → Fault → Failure as a CAUSAL CHAIN (human error creates a fault in code, which causes an observable failure at runtime) — examiners specifically test whether you understand this sequence, not just the three definitions in isolation.

— End of Notes —

Software Engineering | UP LT Grade CS Mains Preparation